

TECHNICAL REQUIREMENTS DOCUMENT

Medical Appointment No-Show Prediction System

Document Type	Technical Requirements Document (TRD)
Related Document	Product Requirements Document (PRD) v1.0
Version	1.0 (Draft)
Status	Proposed
Owner	Engineering / Data Science Team

1. Purpose

This Technical Requirements Document (TRD) translates the Product Requirements Document (PRD) for the Medical Appointment No-Show Prediction System into concrete technical specifications: system architecture, data pipeline, model design, APIs, infrastructure, and quality/testing requirements. It is intended for engineers and data scientists implementing the system.

2. System Architecture

The system is split into two pipelines: an **offline training pipeline** that builds and evaluates the model from historical data, and an **online scoring pipeline** that applies the trained model to upcoming appointments and surfaces risk scores to staff.

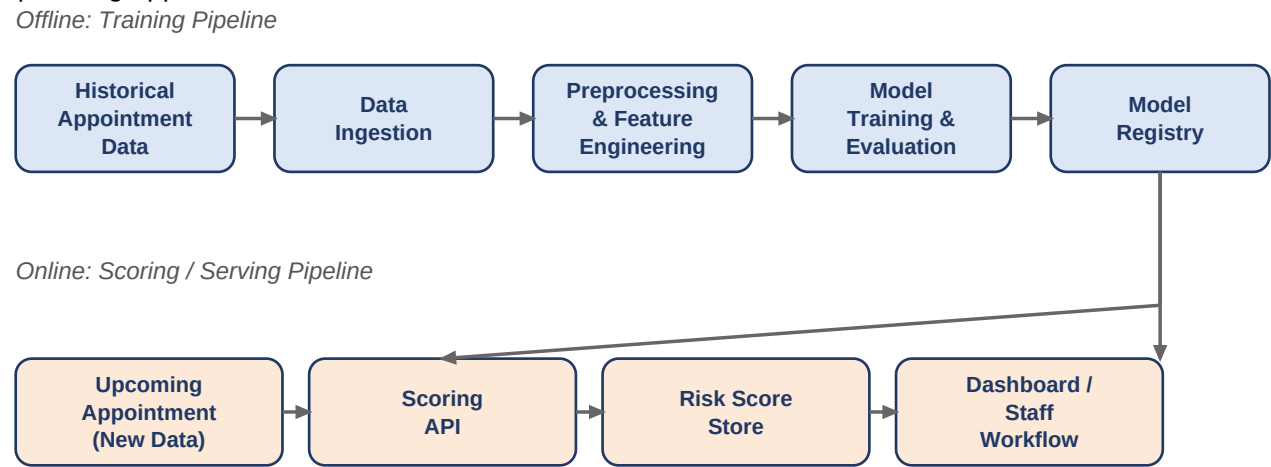


Figure 1. High-level system architecture

2.1 Components

Component	Responsibility
Data Ingestion	Extracts appointment records from source systems (hospital DB / CSV exports) into a staging area.
Preprocessing & Feature Engineering	Cleans data and derives model features (waiting days, weekday, patient history, etc.).
Model Training & Evaluation	Trains candidate algorithms, tunes hyperparameters, and evaluates against holdout data.
Model Registry	Stores versioned trained models and metadata (metrics, training date, feature schema).

Component	Responsibility
Scoring API	Loads the current production model and returns a no-show probability for a given appointment.
Risk Score Store	Persists per-appointment scores for reporting and downstream workflow use.
Dashboard / Staff Workflow	Displays ranked risk lists and analytics charts to clinic staff and admins.

3. Data Pipeline Requirements

3.1 Data Ingestion

- Support batch ingestion from CSV/SQL exports on a scheduled basis (e.g., nightly).
- Validate incoming records against an expected schema; reject/quarantine malformed rows.
- Maintain an audit log of ingestion runs (timestamp, row count, errors).

3.2 Preprocessing

- Deduplicate records on Appointment ID.
- Impute or flag missing values (e.g., missing age, missing SMS status).
- Normalize date fields into a consistent format; compute Waiting Days = Appointment Date – Scheduled Date.
- Encode categorical variables (Gender, Appointment Type, Day of Week) using one-hot or ordinal encoding as appropriate to the chosen model.
- Detect and cap/flag outliers (e.g., negative waiting days, implausible ages).

3.3 Feature Engineering

Feature	Type	Description
waiting_days	int	Days between scheduling and appointment
appointment_dow	categorical	Day of week of the appointment
appointment_month	categorical	Month of the appointment
age_group	categorical	Binned age (e.g., 0-12, 13-19, 20-40, 41-60, 60+)
prior_noshow_rate	float	Patient's historical no-show rate prior to this appointment
sms_received	boolean	Whether a reminder SMS was sent
chronic_condition	boolean	Whether the patient has a chronic condition on record

4. Model Requirements

4.1 Candidate Algorithms

- Logistic Regression (baseline, interpretable).

- Decision Tree.
- Random Forest.
- XGBoost (gradient boosting).
- Support Vector Machine (SVM).

4.2 Training Process

- Split data into training/validation/test sets (e.g., 70/15/15), stratified on the target variable.
- Apply k-fold cross-validation during hyperparameter tuning.
- Address class imbalance (no-shows are typically a minority class) via class weighting or resampling (e.g., SMOTE).
- Log all experiment runs (parameters, metrics, artifacts) for reproducibility.

4.3 Evaluation Metrics

Metric	Purpose
Accuracy	Overall correctness of predictions
Precision	Of predicted no-shows, how many were correct
Recall	Of actual no-shows, how many were correctly flagged (prioritized)
F1-Score	Harmonic mean of precision and recall (prioritized)
Confusion Matrix	Breakdown of true/false positives and negatives
ROC-AUC	Model's ability to discriminate between classes across thresholds

The production model must be selected based on the best Recall/F1 trade-off, not accuracy alone, since failing to flag an actual no-show is more costly than a false alarm.

4.4 Model Output Contract

The model produces a single output per appointment: **no_show_probability** (float, 0.0–1.0). Downstream systems apply a configurable threshold (default 0.5, tunable per clinic) to classify an appointment as high-risk.

5. Scoring API Specification

5.1 Endpoint: Predict No-Show Probability

POST /api/v1/predict

Request body:

```
{
  "appointment_id": "string",
  "patient_id": "string",
  "age": "int",
  "gender": "string",
  "scheduled_date": "YYYY-MM-DD",
  "appointment_date": "YYYY-MM-DD",
  "sms_received": "boolean",
```

```
"chronic_condition": "boolean"
}
```

Response body:

```
{
  "appointment_id": "string",
  "no_show_probability": "float",
  "risk_tier": "low | medium | high",
  "model_version": "string"
}
```

5.2 Endpoint: Batch Scoring

POST /api/v1/predict/batch — accepts a list of appointments and returns scores for all upcoming appointments (used for nightly re-scoring of the schedule).

5.3 API Non-Functional Requirements

- p95 response time under 500ms for single-record scoring.
- Batch endpoint should process at least 10,000 records within 5 minutes.
- API must be versioned (e.g., /api/v1/) to allow non-breaking model upgrades.
- Authentication required (API key or OAuth token) for all endpoints.

6. Data Model (Core Tables)

Table	Key Columns
patients	patient_id (PK), age, gender, chronic_condition
appointments	appointment_id (PK), patient_id (FK), scheduled_date, appointment_date, sms_received, appointment_type, no_show (target)
predictions	appointment_id (FK), no_show_probability, risk_tier, model_version, scored_at
model_registry	model_version (PK), algorithm, training_date, metrics_json, artifact_path

7. Infrastructure and Deployment

Area	Requirement
Environments	Separate dev, staging, and production environments
Model Serving	Containerized scoring service (e.g., Docker) behind the Scoring API
Storage	SQL database for structured data; object storage for model artifacts
Orchestration	Scheduled batch jobs for ingestion, retraining, and nightly re-scoring

Area	Requirement
CI/CD	Automated tests and model validation gates before promoting a new model to production
Visualization	Power BI / Tableau or an embedded Python-based dashboard reading from the Risk Score Store

8. Security and Compliance

- Encrypt patient data at rest and in transit.
- Apply role-based access control (RBAC) — staff see only their clinic's data.
- Mask or exclude direct identifiers from model training features where not needed.
- Maintain audit logs of who accessed predictions and patient data.
- Review against applicable healthcare data privacy regulations before production rollout.

9. Testing Strategy

Test Type	Coverage
Unit Tests	Preprocessing functions, feature engineering logic, API request/response handling
Integration Tests	End-to-end pipeline from ingestion through scoring API response
Model Validation Tests	Minimum thresholds for Recall/F1/ROC-AUC before a model can be promoted
Load Testing	Scoring API under expected peak concurrent request volume
Regression Testing	New model versions compared against the current production model on the same test set

10. Monitoring and Retraining

- Track live prediction distribution and compare against training-time distribution (data drift).
- Recompute evaluation metrics periodically using newly confirmed appointment outcomes.
- Trigger alerts if performance drops below defined thresholds.
- Support scheduled or trigger-based retraining using the latest historical data.

11. Open Technical Questions

- What is the source system for historical appointment data, and what integration method is available (direct DB access, export files, API)?
- What volume of historical data is currently available for initial training?
- Is there an existing authentication/identity system the Scoring API should integrate with?
- What is the required data retention period for predictions and patient records?